

C - Properties of Array

Arrays are a very important data structure in C. Use of array in C program makes it easier to handle large amount of data. Arrays have a number of advantages over singular variables as a result of their number of properties. Most of the important properties of array are a result of its composition – that an array is a collection of values of same **data type**, and in a continuous block of memory.

Array properties may change in the different programming languages. In C programming language, the following are the main properties of arrays:

- Collection of Same Data Type
- Contiguous Memory Allocation
- Fixed Size
- Length Depends on Type
- Indexing
- Pointer Relationship
- Lower and Upper Bounds
- Multi-dimensional Array
- Implementation of Complex Data Structures

Let us discuss each property in detail.

Collection of Same Data Type

All elements of an **array** must be of the same data type. This ensures consistent access and operations on the data.

If an array is declared as follows –

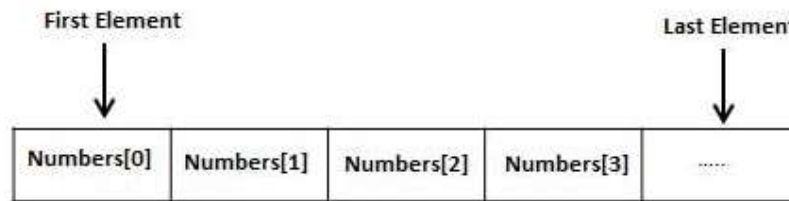
```
int arr[] = {50, 67.55, "hello", 21};
```

Compiler issues a warning –

```
initialization of 'int' from 'char *' makes integer from pointer without a cast  
[-Wint-conversion]
```

Contiguous Memory Allocation

All elements of an array are stored in contiguous memory locations, meaning they occupy a block of memory next to each other. This allows for efficient random access and memory management.



Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Fixed Size

The size of an array is fixed at the time of declaration and cannot be changed during the program's execution. This means you need to know the maximum number of elements you need beforehand. In C, an array cannot have a size defined in terms of a variable.

//This is accepted

```
#define SIZE = 10
int arr[SIZE];
```

//This is also accepted

```
const SIZE = 10;
int arr[SIZE];
```

//This is not accepted

```
int SIZE = 10;
int arr[SIZE];
```

//The size must be an integer. This will give error

```
float num[10.5] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
```

Length Depends on Type

Since an array can store all the elements of same type, the total memory occupied by it depends on the data type.

Example


[Open Compiler](#)

```
#include<stdio.h>

int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    int size = sizeof(num) / sizeof(int);
    printf("element at lower bound num[0]: %d \n", num[0]);
    printf("at upper bound: %d byte \n", num[size-1]);
    printf("length of int array: %ld \n", sizeof(num));
    double nm[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    size = sizeof(nm) / sizeof(double);
    printf("element at lower bound nm[0]: %f \n", nm[0]);
    printf("element at upper bound: %f \n", nm[size-1]);
    printf("byte length of double array: %ld \n", sizeof(nm));

    return 0;
}
```

Output

```
element at lower bound num[0]: 50
at upper bound: 51 byte
length of int array: 40
element at lower bound nm[0]: 50.000000
element at upper bound: 51.000000
byte length of double array: 80
```

Indexing

Each element in an array has a unique index, starting from 0. You can access individual elements using their index within square brackets. Usually, array is traversed with a **for loop** running over its length and using the loop variable as the index.

Example

```
#include <stdio.h>
```

```
int main() {
    int a[] = {1,2,3,4,5};
    int i;

    for (i=0; i<4; i++){
        printf("a[%d]: %d \n", i, a[i]);
    }
    return 0;
}
```

Pointer Relationship

The name of an array is equivalent to a constant **pointer** to its first element. This lets you use array names and pointers interchangeably in certain contexts.

Example

</>

[Open Compiler](#)

```
#include <stdio.h>
int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    printf("num[0]: %d Address of 0th element: %d\n", num[0], &num[0]);
    printf("Address of array: %d", num);
    return 0;
}
```

Output

```
num[0]: 50 Address of 0th element: 6422000
Address of array: 6422000
```

Lower and Upper Bounds

Each element in an array is identified by an index starting with 0. The lower bound of an array is the index of its first element, which is always 0. The last element in the array is identified by the array size - 1 as its index.

Example



Open Compiler

```
#include <stdio.h>
int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    int size = sizeof(num) / sizeof(int);
    printf("element at lower bound num[0]: %d at upper bound: %d Size of array: %d",
        num[0], num[size-1], size);
    return 0;
}
```

Output

```
element at lower bound num[0]: 50 at upper bound: 51 Size of array: 10
```

Multi-dimensional Array

The array is declared with one value of size in square brackets , it is called one dimensional array. In a one dimensional array, each element is identified by its index or subscript. In C, you can declare with more indices to simulate a two, three or **multidimensional array**.

Example

For example, following is the example of a two-dimensional array –

```
int a[3][3] = { {1, 2, 3}, {11, 22, 33}, {111, 222, 333}};
```

You can think of a one dimensional array as a list, and a two dimensional array as a table or a matrix. Theoretically, there is no limit to the number of dimensions of an array, but in practice, two-dimensional arrays are used in design of spreadsheets, databases etc.

Implementation of Complex Data Structures

We can use an array in the construction of a struct data type to implement data structures such as stack, linked list and trees.

Example

```
typedef struct stack {  
    int top;  
    int arr[10];  
} Stack;
```

Thus, array is an important tool in the armoury of the programmer, as it can be used for different applications. The concept of array in C is implemented by many subsequent programming languages such as C++, C#, Java etc.